

# SFC Placement System User Manual

## Table of Contents

|                                               |    |
|-----------------------------------------------|----|
| Introduction.....                             | 2  |
| System Architecture.....                      | 2  |
| Installation .....                            | 3  |
| Prerequisites.....                            | 3  |
| Setup Steps.....                              | 3  |
| Configuration.....                            | 3  |
| Configuration Files .....                     | 3  |
| System Configuration .....                    | 4  |
| Basic Usage.....                              | 5  |
| Part 1: VNF Placement.....                    | 5  |
| Example output: .....                         | 6  |
| Part 2: VNF Placement with Redundancy .....   | 6  |
| Example output: .....                         | 7  |
| Algorithms.....                               | 7  |
| VNF Placement Algorithms (Part 1).....        | 7  |
| Backup VNF Placement Algorithms (Part 2)..... | 8  |
| Input/Output Files.....                       | 8  |
| Input Files .....                             | 8  |
| Output Files .....                            | 8  |
| System Components.....                        | 9  |
| Core Classes.....                             | 9  |
| Setup Components .....                        | 10 |
| Troubleshooting.....                          | 10 |
| Common Issues .....                           | 10 |
| Advanced Usage .....                          | 10 |
| Custom Topology Generation.....               | 10 |
| Custom SFC Creation.....                      | 11 |
| Performance Tuning.....                       | 11 |

# Introduction

The SFC Placement System is a tool for optimizing the placement of Virtual Network Functions (VNFs) in network infrastructures. It addresses key challenges in modern networks by considering multiple optimization criteria:

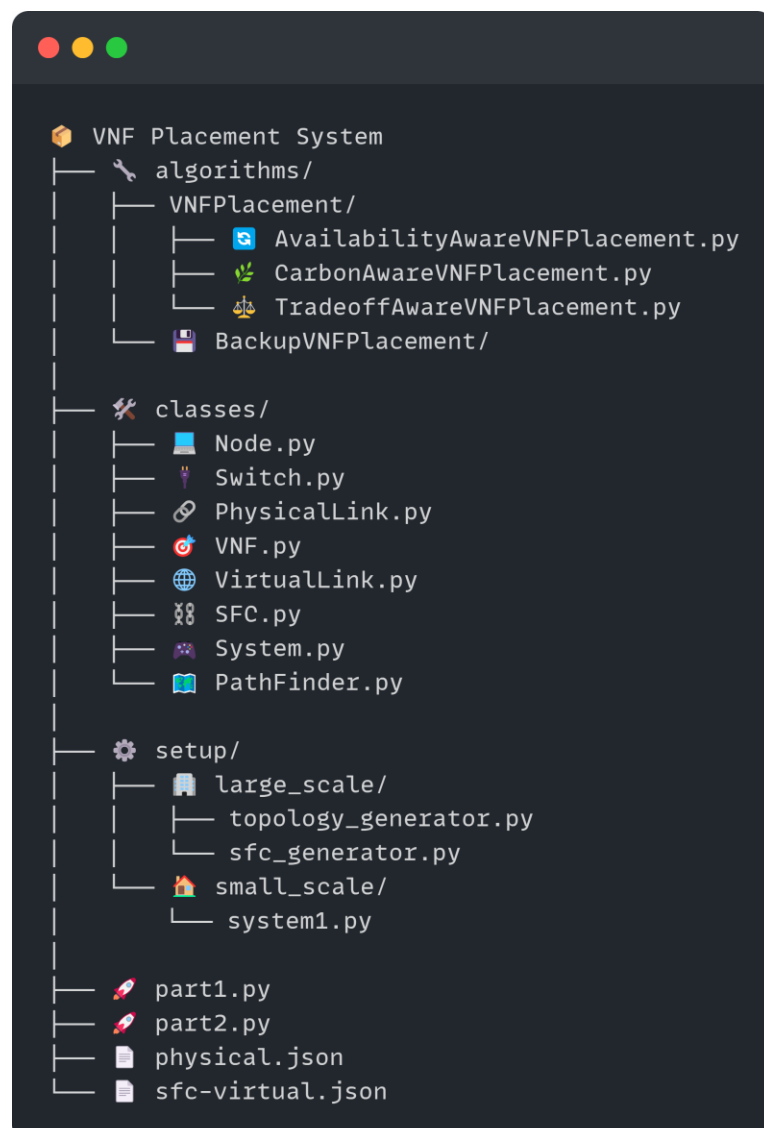
- **Availability:** Ensures high service reliability
- **Carbon Footprint:** Minimizes environmental impact
- **Latency:** Optimizes performance
- **Trade-offs:** Balances competing objectives

The system supports two main operations:

1. Primary VNF placement (Part 1)
2. VNF placement with redundancy optimization (Part 2)

## System Architecture

The SFC Placement System follows a layered architecture:



The architecture consists of:

- **Physical Infrastructure Layer:** Nodes, switches, and physical links
- **Virtual Service Layer:** VNFs, virtual links, and service function chains (SFCs)
- **Optimization Layer:** Placement algorithms and backup strategies
- **Configuration Layer:** System settings and topology definitions

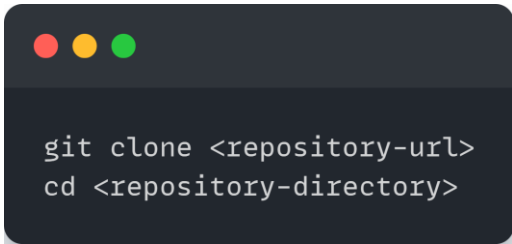
## Installation

### Prerequisites

- Python 3.7 or higher
- Required Python packages
- Git (for repository cloning)
- 4GB RAM minimum (8GB recommended for large topologies)

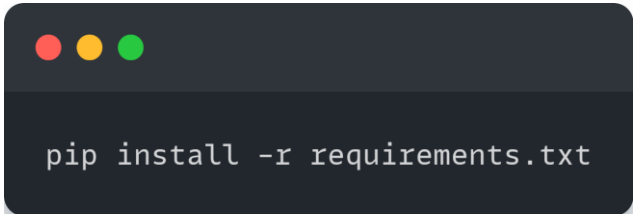
### Setup Steps

1. Clone the repository:



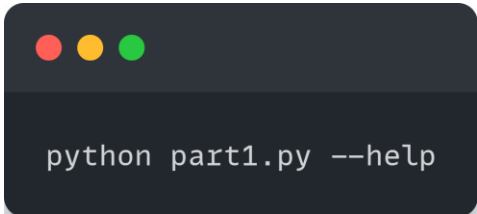
```
git clone <repository-url>
cd <repository-directory>
```

2. Install required dependencies:



```
pip install -r requirements.txt
```

3. Verify installation:



```
python part1.py --help
```

## Configuration

### Configuration Files

The system uses JSON configuration files:

1. **physical.json**: Defines the physical infrastructure

```
{
  "nodes": [
    {"id": "n1", "cpu": 8, "ram": 16, "storage": 100, "availability": 0.999, "carbon_factor": 0.8},
    {"id": "n2", "cpu": 16, "ram": 32, "storage": 200, "availability": 0.995, "carbon_factor": 1.2}
  ],
  "switches": [
    {"id": "s1"},
    {"id": "s2"}
  ],
  "links": [
    {"id": "l1", "source": "n1", "target": "s1", "bandwidth": 10, "delay": 5},
    {"id": "l2", "source": "s1", "target": "s2", "bandwidth": 10, "delay": 2},
    {"id": "l3", "source": "s2", "target": "n2", "bandwidth": 10, "delay": 5}
  ]
}
```

2. **virtual.json**: Defines the service function chains

```
{
  "sfcs": [
    {
      "id": "sfc1",
      "vnfs": [
        {"id": "VNF_0_0", "cpu": 2, "ram": 4, "storage": 10},
        {"id": "VNF_0_1", "cpu": 4, "ram": 8, "storage": 20}
      ],
      "virtual_links": [
        {"id": "vl1", "source": "VNF_0_0", "target": "VNF_0_1", "bandwidth": 5}
      ]
    }
  ]
}
```

## System Configuration

The system is configured through the **config.py** file, where you can set:

- **EMBEDDING\_POLICY**: Determines VNF placement strategy
  - **AAE**: Availability-Aware Embedding
  - **CAE**: Carbon-Aware Embedding
  - **TAE**: Trade-off Aware Embedding (balances multiple objectives)
- **ALLOCATION\_POLICY**: Determines redundancy optimization strategy (for Part 2)
  - **SA**: Simulated Annealing
  - **GA**: Genetic Algorithm
  - **PSO**: Particle Swarm Optimization
- Systems Parameters
- Optimization Weights

```
# VNF Placement Policy Options
# "AAE" - Availability-Aware Embedding
# "CAE" - Carbon-Aware Embedding
# "TAE" - Trade-off Aware Embedding
EMBEDDING_POLICY = "TAE"

# Backup VNF Placement Policy Options (Part 2 only)
# "SA" - Simulated Annealing
# "GA" - Genetic Algorithm
# "PSO" - Particle Swarm Optimization
ALLOCATION_POLICY = "PSO"

# System Parameters
MIN_AVAILABILITY = 0.9999
MAX CARBON FOOTPRINT = 5000
MAX_LATENCY = 100

# Optimization Weights
AVAILABILITY_WEIGHT = 0.4
CARBON_WEIGHT = 0.6
```

## Basic Usage

### Part 1: VNF Placement

Part 1 focuses on the initial placement of VNFs without redundancy:

```
python part1.py
```

#### Options:

- `--policy <POLICY>`: Override the embedding policy from [config.py](#)
- `--physical <FILE>`: Specify a custom physical topology file
- `--virtual <FILE>`: Specify a custom virtual topology file
- `--output <DIR>`: Specify the output directory

#### Example:

```
python part1.py --policy TAE --physical custom_physical.json --virtual custom_virtual.json --output results
```

This will:

1. Load system topology from configuration
2. Apply the selected embedding policy (from config.py)
3. Calculate system metrics (availability, carbon footprint, latency)
4. Output placement results

Example output:

```
System: system1
=====
SFC: sfc1
=====
VNFs: 3
Virtual Links: 2
Availability: 0.99345
Latency: 23.5
-----
VNF Mapping:
-----
VNF: VNF_0_0 → Node: n1, Availability: 99.90 %, Delay: 4.5 ms
VNF: VNF_0_1 → Node: n2, Availability: 99.85 %, Delay: 5.2 ms
VNF: VNF_0_2 → Node: n3, Availability: 99.78 %, Delay: 6.1 ms
Availability: 0.99345
Carbon Footprint: 2385.6
Latency: 23.5
Redundancy: 0
Solution: [0, 0, 0]
Objective: 0.79654
```

## Part 2: VNF Placement with Redundancy

Part 2 extends Part 1 by adding redundancy optimization:

```
python part2.py
```

Options:

- `--policy <POLICY>`: Override the embedding policy
- `--allocation <POLICY>`: Override the allocation policy
- `--physical <FILE>`: Specify a custom physical topology file
- `--virtual <FILE>`: Specify a custom virtual topology file
- `--output <DIR>`: Specify the output directory

#### Example:

```
python part2.py --policy TAE --allocation GA --physical custom_physical.json --virtual custom_virtual.json
```

This will:

1. Perform initial VNF placement (as in Part 1)
2. Apply the selected allocation policy to optimize redundancy
3. Place backup VNFs
4. Calculate enhanced system metrics
5. Output results including backup placements

#### Example output:

```
VNF: VNF_0_0 → Node: n1, Availability: 99.90 %, Delay: 4.5 ms
VNF: VNF_0_1 → Node: n2, Availability: 99.85 %, Delay: 5.2 ms
VNF: VNF_0_2 → Node: n3, Availability: 99.78 %, Delay: 6.1 ms
VNF: VNF_0_0-backup-0 → Node: n2, Availability: 99.85 %, Delay: 5.0 ms
VNF: VNF_0_1-backup-0 → Node: n1, Availability: 99.90 %, Delay: 4.8 ms
VNF: VNF_0_2-backup-0 → Node: n2, Availability: 99.85 %, Delay: 5.2 ms
Availability: 0.99992
Carbon Footprint: 3651.2
Latency: 25.8
Redundancy: 3
Solution: [1, 1, 1]
Objective: 0.87321
```

## Algorithms

### VNF Placement Algorithms (Part 1)

The system implements three primary VNF placement strategies:

1. **Availability-Aware VNF Placement (AAE)**
  - a. **Objective:** Maximize service availability
  - b. **Approach:** Prioritizes nodes with higher availability values
  - c. **Use Case:** Mission-critical applications with strict SLAs
  - d. **Implementation:** AvailabilityAwareVNFPlacement.py

## 2. Carbon-Aware VNF Placement (CAE)

- a. **Objective:** Minimize carbon footprint
- b. **Approach:** Prioritizes nodes with lower carbon factors
- c. **Use Case:** Environmentally conscious deployments
- d. **Implementation:** CarbonAwareVNFPlacement.py

## 3. Trade-off Aware VNF Placement (TAE)

- a. **Objective:** Balance availability, carbon footprint, and latency
- b. **Approach:** Multi-objective optimization using weighted metrics
- c. **Use Case:** General-purpose deployments with balanced requirements
- d. **Implementation:** TradeoffAwareVNFPlacement.py

# Backup VNF Placement Algorithms (Part 2)

For redundancy optimization, the system implements:

## 1. Simulated Annealing (SA)

- a. **Approach:** Probabilistic optimization with temperature cooling
- b. **Advantages:** Good for escaping local optima, works well with discrete solution spaces
- c. **Parameters:** Initial temperature, cooling rate, iterations
- d. **Implementation:** SABackupVNFPlacement.py

## 2. Genetic Algorithm (GA)

- a. **Approach:** Evolutionary algorithm with selection, crossover, and mutation
- b. **Advantages:** Efficient for complex solution spaces with many variables
- c. **Parameters:** Population size, generations, mutation rate, selection rate
- d. **Implementation:** GeneticBackupVNFPlacement.py

## 3. Particle Swarm Optimization (PSO)

- a. **Approach:** Swarm intelligence with particle position and velocity updates
- b. **Advantages:** Fast convergence, handles continuous solution spaces well
- c. **Parameters:** Swarm size, inertia weight, cognitive/social coefficients
- d. **Implementation:** PSOBakupVNFPlacement.py

# Input/Output Files

## Input Files

### 1. Physical Topology (physical.json)

- a. Defines physical infrastructure (nodes, switches, links)
- b. Specifies node resources, availability, and carbon factors
- c. Specifies link bandwidth and delay

### 2. Virtual Topology (virtual.json)

- a. Defines service function chains (SFCs)
- b. Specifies VNF resource requirements
- c. Specifies virtual links between VNFs

## Output Files

The system generates outputs in the output/{EMBEDDING\_POLICY}-{ALLOCATION\_POLICY}/ directory:

1. **Placement Results (placements.txt)**
  - a. VNF to node mappings
  - b. System metrics (availability, carbon footprint, latency)
  - c. Optimization objective value
2. **Backup Placement Results (placements-backup.txt)**
  - a. Primary and backup VNF mappings
  - b. Enhanced system metrics with redundancy
  - c. Backup solution encoding
3. **System State (system\_simulator.pkl)**
  - a. Serialized system state for visualization
  - b. Can be loaded for further analysis
4. **Updated Topologies (physical.json, virtual.json)**
  - a. Physical topology with updated resource utilization
  - b. Virtual topology with deployment information
5. **Metrics Log (metrics.csv)**
  - a. Detailed metrics for each optimization run
  - b. Useful for comparative analysis

## System Components

### Core Classes

1. **System (System.py)**
  - a. Main controller managing the entire deployment
  - b. Coordinates between physical and virtual components
  - c. Methods:
    - i. `place_vnfs()`: Executes VNF placement
    - ii. `place_backup_vnfs()`: Executes backup VNF placement
    - iii. `calculate_metrics()`: Computes system metrics
    - iv. `export_results()`: Saves placement results
2. **Node (Node.py)**
  - a. Represents a physical server with resources
  - b. Properties: CPU, RAM, storage, availability, carbon factor
  - c. Methods:
    - i. `allocate_resources()`: Assigns resources to VNFs
    - ii. `release_resources()`: Frees resources when VNFs are removed
    - iii. `calculate_load()`: Computes current resource utilization
3. **VNF (VNF.py)**
  - a. Represents a Virtual Network Function
  - b. Properties: Resource requirements, hosting node, backup status
  - c. Methods:
    - i. `calculate_availability()`: Computes VNF availability
    - ii. `calculate_carbon_footprint()`: Computes VNF carbon impact
    - iii. `calculate_latency()`: Computes VNF processing delay
4. **SFC (SFC.py)**
  - a. Represents a Service Function Chain
  - b. Contains ordered collection of VNFs and virtual links

- c. Methods:
  - i. `calculate_end_to_end_availability()`: Computes chain availability
  - ii. `calculate_total_carbon_footprint()`: Computes chain carbon impact
  - iii. `calculate_end_to_end_latency()`: Computes chain latency

#### 5. **PathFinder (PathFinder.py)**

- a. Computes network paths between nodes
- b. Implements shortest path and k-shortest paths algorithms
- c. Methods:
  - i. `find_shortest_path()`: Finds optimal path between two nodes
  - ii. `find_k_shortest_paths()`: Finds k alternative paths
  - iii. `calculate_path_metrics()`: Computes path availability, delay

## Setup Components

#### 1. **Topology Generator (topology\_generator.py)**

- a. Creates different network topologies
- b. Supports fat-tree and custom topologies
- c. Methods:
  - i. `generate_fat_tree()`: Creates a fat-tree topology
  - ii. `generate_random_topology()`: Creates a random topology

#### 2. **SFC Generator (sfc\_generator.py)**

- a. Creates service function chains
- b. Configurable VNF requirements and chain complexity
- c. Methods:
  - i. `generate_sfc()`: Creates a single SFC
  - ii. `generate_random_sfcs()`: Creates multiple random SFCs

## Troubleshooting

### Common Issues

1. Resource constraints violated
  - a. Symptom: Placement fails with resource constraint errors
  - b. Solution: Increase node resources or reduce VNF requirements
2. Availability constraints not met
  - a. Symptom: System availability below minimum threshold
  - b. Solution: Lower [min\\_availability](#) or add more redundancy
3. Carbon footprint exceeds limits
  - a. Symptom: Carbon footprint exceeds maximum threshold
  - b. Solution: Adjust [max\\_carbon\\_footprint](#) or use more efficient nodes

## Advanced Usage

### Custom Topology Generation

You can generate custom topologies using the provided generators:

```
from setup.large_scale.topology_generator import generate_fat_tree

# Generate a fat-tree topology with k=4
nodes, edge_switches, aggr_switches, core_switches, physical_links = generate_fat_tree(k=4)
```

## Custom SFC Creation

Create custom service function chains:

```
from setup.large_scale.sfc_generator import generate_sfc

# Generate an SFC with 3 VNFs
sfc = generate_sfc(sfc_id=0, num_vnfs=3)
```

## Performance Tuning

For large-scale deployments, tune algorithm parameters:

```
# Genetic Algorithm with optimized parameters
allocation = GeneticBackupVNFPlacement(
    policy=policy,
    population_size=100, # Larger population
    generations=75,      # More generations
    mutation_rate=0.15,  # Higher mutation rate
    selection_rate=0.3    # Higher selection rate
)
```